

Capacity & the Design Method

The close of Part VIII (capacity) and all of Part IX — turning knowledge into a repeatable method.

51 Capacity Planning — turning estimates into provisioning decisions

52 The System Design Interview — a repeatable framework

53 Estimation & Back-of-the-Envelope Math — sizing a system in your head

54 Communicating & Trade-offs — the senior-engineer skill

Chapter 51 completes operations; Chapters 52-54 turn the whole book into a method for designing and defending systems.

CHAPTER 51

Capacity Planning

Part VIII · Infrastructure & Operations — turning estimates into provisioning decisions.

LEARNING OBJECTIVES

- Turn traffic and data estimates into concrete resource needs.
- Provision for peak, failure, and growth — not average.
- Explain headroom and why you never plan for 100% utilization.
- Combine autoscaling with proactive capacity planning.

REAL-WORLD ANALOGY — SIZING A RESTAURANT

Opening a restaurant, you plan tables, kitchen, and staff for the busy Friday night — not the quiet Tuesday afternoon — plus a little slack for an unexpectedly full house. Too few tables and you turn customers away; too many and you pay rent on empty space. Capacity planning sizes a system the same way: for the peak, with a margin.

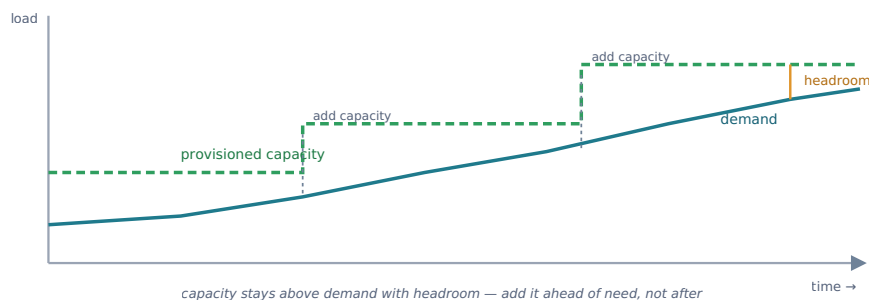
51.1 The problem

Chapter 5 taught back-of-the-envelope estimation and Chapter 10 taught scaling. Capacity planning joins them: converting expected **load** into concrete decisions — how many servers, how much storage, how much headroom — and knowing *when* to add more. It closes Part VIII.

51.2 The inputs

You need: **traffic** (average *and* peak QPS, with percentiles from Chapter 5), **data volume and growth rate**, the **per-request resource cost** (CPU, memory, I/O), and **latency targets**. The **peak-to-average ratio** is critical — you must provision for the peak, which can be several times the average.

51.3 Provision for peak, failure, and growth



KEY INSIGHT — NEVER PLAN FOR 100% UTILIZATION

Recall the utilization curve (Chapter 5): as a resource nears saturation, latency explodes. So target a comfortable peak utilization (often 50–70%) and keep **headroom** for three things:

traffic **spikes, failures** (lose a node or an AZ and still serve — N+1 redundancy, Chapter 34), and **growth**. Planning to run full is planning to fall over.

51.4 Autoscaling plus planning

Autoscaling (Chapter 10) handles short-term fluctuations automatically, but it isn't a substitute for planning: instances take time to start, quotas and reserved capacity (Chapter 49) must be arranged ahead, and big events (a launch, a sale) need deliberate pre-provisioning. **Load testing** finds your real per-instance limits so estimates match reality. Capacity planning is **proactive** — add capacity *before* you hit the wall.

Common mistakes

WATCH OUT

- Provisioning for average load, then falling over at peak.
- Targeting ~100% utilization with no headroom for spikes or failures.
- Assuming autoscaling is instant and infinite (start-up lag, quotas).
- Never load testing, so capacity numbers are guesses.
- Ignoring data **growth** and running out of storage.

Interview questions

1. How do you turn traffic estimates into a server count?
2. Why provision for peak and failure rather than average?
3. Why keep headroom, and how much?
4. How do autoscaling and capacity planning complement each other?

Hands-on exercises

1. Given 10,000 peak QPS and 500 QPS/instance, size the fleet with N+1 and 60% target utilization.
2. Estimate 1 year of storage for 1M new 2 KB records/day.
3. Explain why you provision for peak, using a peak-to-average ratio of 5.
4. Describe a load test to find an instance's safe QPS.

Mini project

PLAN AND VALIDATE CAPACITY

Pick a service and estimate its peak QPS and data growth. Load-test one instance to find its safe throughput, then compute the fleet size with headroom and N+1 redundancy. Configure autoscaling around that baseline and run a spike test to confirm the headroom absorbs it before autoscaling catches up.

| Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE CAPACITY PLAN

From your Chapter 5 estimates, size each tier (app, cache, database, queue) for **peak** load with headroom and N+1 across AZs (Chapter 48). Set autoscaling baselines and reserved capacity (Chapter 49), plan storage growth with tiering, and define a pre-provisioning plan for launch spikes. Load-test to validate the numbers.

| Chapter summary

- Capacity planning converts **load estimates** into concrete provisioning: servers, storage, headroom.
- Provision for **peak, failure, and growth** — never average.
- Keep **headroom** (target 50–70% peak utilization); never plan for full saturation.
- **Autoscaling** handles short-term; **proactive planning** and **load testing** handle the rest.

CHAPTER 52

The System Design Interview

Part IX · Design Method & Interview — a repeatable framework.

LEARNING OBJECTIVES

- Understand what a design interview actually evaluates.
- Apply a repeatable 7-step framework to any prompt.
- Avoid the common pitfalls.
- Treat the interview as a structured conversation.

REAL-WORLD ANALOGY — AN ARCHITECT’S CONSULTATION

A client says “design me a building.” A good architect doesn’t start drawing walls — they ask who will use it, how many, what budget, what constraints; sketch a rough massing; discuss trade-offs; and iterate. The design interview works the same way: it tests how you **think through** an open problem, not whether you memorized an answer.

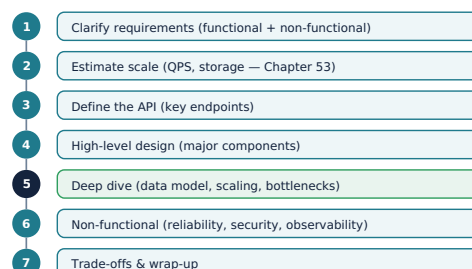
52.1 Part IX begins

Parts I–VIII built the toolkit. Part IX assembles it into a **method**: a repeatable way to approach any “design X” problem, estimate its scale, and communicate the design. It starts with the interview framework — equally useful for real design work.

52.2 What it evaluates

An open prompt (“design Twitter”) has **no single right answer**. Interviewers assess your **structured approach**, ability to **clarify requirements**, **estimation**, **breadth** of knowledge with **depth** in a few areas, **trade-off reasoning**, and **communication** under ambiguity.

52.3 A repeatable framework



Walk these in order, but loop back as you learn more. Steps 1–2 anchor everything in requirements and scale; step 5 is where you show depth on the parts that matter most.

52.4 Common pitfalls

WATCH OUT

- Jumping to a solution before clarifying requirements.
- Skipping estimation, so the design isn't sized to the scale.
- Over-engineering (microservices, multi-region) when not warranted.
- Going silent — not thinking aloud — or getting stuck in one corner.
- Never discussing trade-offs or alternatives.

Interview questions

1. What does a design interview actually evaluate?
2. Walk through your framework for an open prompt.
3. Why clarify requirements and estimate scale first?
4. Where do you demonstrate depth versus breadth?

Hands-on exercises

1. List the clarifying questions you'd ask for "design a chat app."
2. Separate functional from non-functional requirements for a URL shortener.
3. Apply the 7 steps at a high level to "design a pastebin."
4. Identify where you'd go deep for "design a news feed."

Mini project

RUN A MOCK DESIGN

Take a prompt ("design a link shortener") and work the full 7-step framework on paper, timing yourself: requirements, estimates, API, high-level diagram, one deep dive, non-functional concerns, and trade-offs. Then critique your own answer for pitfalls — did you clarify first, size it, and discuss alternatives?

Large project (capstone continues)

YOUR SOCIAL PLATFORM — AS AN INTERVIEW

Present your entire capstone through the framework: the requirements you clarified, your scale estimates, the API, the high-level architecture (LB, services, cache, DB, queue), deep dives (feed generation, sharding), and the non-functional story (reliability, security, observability). This is your platform retold as a structured design walkthrough.

Chapter summary

- The design interview evaluates **structured thinking, breadth/depth, trade-offs, and communication** — not one right answer.

- Use a repeatable framework: **clarify → estimate → API → high-level → deep dive → non-functional → trade-offs**.
- Anchor in **requirements and scale** before designing; show depth where it matters.
- Avoid jumping ahead, over-engineering, and silence; treat it as a conversation.

CHAPTER 53

Estimation & Back-of-the-Envelope Math

Part IX · Design Method & Interview — sizing a system in your head.

LEARNING OBJECTIVES

- Explain why estimation drives architectural decisions.
- Recall the key numbers and a simple method.
- Work a full example from users to QPS, storage, and bandwidth.
- Focus on order of magnitude, not precision.

REAL-WORLD ANALOGY — A BUILDER'S MATERIAL ESTIMATE

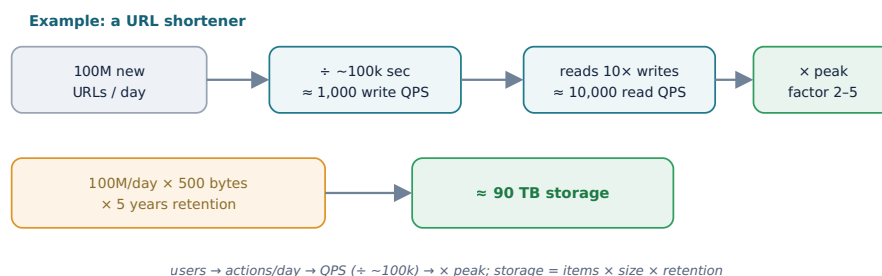
Before construction, a builder roughly estimates the concrete, steel, and bricks — not to the gram, but closely enough to order trucks and size the crew. Get it 10× wrong and the project fails; get the ballpark right and it proceeds. Estimation in system design is exactly this: rough numbers that decide the shape of the build.

53.1 The problem

Architecture depends entirely on scale: 1,000 users and 1 billion users need completely different designs (single database vs sharded, cache or not, CDN or not). **Estimation** turns vague requirements into the numbers that drive those decisions — a core skill (deepening Chapter 5).

53.2 Numbers and method

Keep a few anchors: a day is about **100,000 seconds** ($86,400 \approx 10^5$); powers of ten for KB/MB/GB/TB; rough item sizes (a text post $\approx$ a few hundred bytes, an image $\approx$ 1 MB). Then:



53.3 The steps

Estimate **users** (DAU) × **actions per user** → actions/day; divide by ~100,000 for **average QPS**; multiply by a **peak factor** (2-10×) for peak QPS. **Storage** = items/day × size × retention. **Bandwidth** = QPS × payload size. **Cache memory** $\approx$ the hot fraction (80/20 rule) × item size. Always separate **reads from writes** and **average from peak**.

53.4 Focus on magnitude

KEY INSIGHT — YOU'RE CHOOSING AN ARCHITECTURE, NOT DOING ACCOUNTING

State your assumptions, round aggressively to powers of ten, and aim for the right **order of magnitude**. The number itself doesn't matter — what matters is whether it says “one database is fine” or “you must shard,” “serve from memory” or “add a CDN.” Precision is a distraction; the ballpark drives the design.

Common mistakes

WATCH OUT

- Chasing false precision instead of order of magnitude.
- Not stating assumptions (so the interviewer can't follow).
- Forgetting the peak factor and sizing for average.
- Lumping reads and writes together when they differ 10×.

Interview questions

1. Why does estimation drive architecture?
2. How do you convert daily actions into average and peak QPS?
3. Estimate 5-year storage for 100M items/day at 500 bytes.
4. Why is order of magnitude enough?

Hands-on exercises

1. Estimate QPS and storage for a photo app: 10M DAU, 2 uploads/user/day, 1 MB each.
2. Compute cache memory for the hot 20% of 1B 1 KB objects.
3. Estimate bandwidth for 50,000 read QPS of 2 KB responses.
4. Show how a 5× peak factor changes your server count.

Mini project

ESTIMATE THREE SYSTEMS

Quickly estimate the scale of three systems (a URL shortener, a chat app, a video platform): DAU, read/write QPS with peak, storage over 5 years, and bandwidth. For each, note the one number that forces a key decision (shard? cache? CDN?). Keep every figure to one significant digit and state assumptions.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE NUMBERS, REVISITED

Refine your capstone estimates into a clean one-page sheet: DAU, posts and reads per day, average and peak QPS, storage growth over 5 years, cache size for the hot feed,

and bandwidth. Draw the line from each number to a design decision you made in earlier chapters (sharding, caching, CDN, queue).

Chapter summary

- Estimation converts vague requirements into the **numbers that decide architecture**.
- Method: users × actions → actions/day → ÷ ~**100,000** for avg QPS → × **peak factor**; storage = items × size × retention.
- Separate **reads/writes** and **average/peak**; state assumptions.
- Aim for **order of magnitude** — it drives shard/cache/CDN decisions, not accounting.

CHAPTER 54

Communicating & Trade-offs

Part IX · Design Method & Interview — the senior-engineer skill.

LEARNING OBJECTIVES

- Recognize that every design decision is a trade-off.
- Communicate a design clearly and drive the discussion.
- Justify each decision against the requirements.
- See trade-off reasoning as the core of system design.

REAL-WORLD ANALOGY — A DOCTOR EXPLAINING OPTIONS

A good doctor doesn't just declare a treatment — they explain the diagnosis, lay out the options, name the risks and benefits of each, and justify the recommendation for *your* situation. That reasoning, communicated clearly, is what earns trust. Senior engineers are set apart the same way: not by knowing patterns, but by explaining *why* a choice fits.

54.1 The problem

This chapter closes Part IX with the skill that distinguishes senior and staff engineers: **communicating reasoning** and **navigating trade-offs**. Knowing the patterns is necessary but not sufficient — there are no free lunches, and every decision sacrifices something.

54.2 There are no right answers, only trade-offs

The whole book is a set of recurring tensions. Name the trade-off explicitly whenever you choose:



Consistency vs availability (Chapter 20), latency vs consistency (Chapter 21), simplicity vs scalability (Chapter 29), cost vs reliability (Chapter 49), build vs buy. Every design *slides* along these; your job is to place the slider deliberately and say why.

54.3 Communicating a design

Think aloud so your reasoning is visible; **structure** the talk with the framework (Chapter 52); go **high-level first, then drill down**; draw **diagrams**; define terms; and state assumptions. Drive the discussion: ask clarifying questions, propose a design *and* critique it

yourself, acknowledge alternatives (“we could also do X, but...”), handle pushback gracefully, and admit unknowns honestly.

54.4 Justifying decisions

KEY INSIGHT — TIE EVERY CHOICE BACK TO A REQUIREMENT

For each major decision, state the **options**, the **trade-off**, and **why** you chose given the requirements. For example: “Because payments need strong consistency, I’ll use a relational database with ACID, accepting lower write throughput and adding read replicas for scale.” A decision anchored to a requirement is defensible; a decision stated as a fact is not. This is the whole discipline in one habit.

54.5 The thesis of the book

KEY INSIGHT — SYSTEM DESIGN IS REASONED TRADE-OFFS UNDER CONSTRAINTS

Every part of this book has been a trade-off in disguise. Mastery is not memorizing architectures — it is knowing the patterns, **judging which fits** a given set of requirements and constraints, and **explaining why** clearly. That judgment, communicated well, is what makes a staff engineer.

Common mistakes

WATCH OUT

- Presenting a design as the only option, with no alternatives or trade-offs.
- Silent designing — the interviewer can’t read your mind.
- Decisions untethered from requirements (“I’d use Kafka” — why?).
- Defensiveness under pushback instead of engaging with it.
- Pretending to know something instead of reasoning openly.

Interview questions

1. Why are there no right answers in system design, only trade-offs?
2. How do you communicate a design effectively?
3. How do you justify a major decision?
4. Give an example of tying a choice to a requirement.

Hands-on exercises

1. Name the trade-off in choosing SQL over NoSQL for a given feature.
2. Justify a caching decision by tying it to a latency requirement.
3. Practice proposing a design and then critiquing your own choice.
4. Rewrite “I’d shard the database” as a requirement-anchored justification.

Mini project

DEFEND A DESIGN OUT LOUD

Take one of your earlier designs and present it to a friend (or recording) for ten minutes. For every major component, state the alternatives, the trade-off, and the requirement that justified your choice. Have them push back on two decisions and practice engaging with the pushback rather than defending reflexively.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE TRADE-OFF LOG

Write a one-page trade-off log for your capstone: for each major decision (SQL vs NoSQL, sync vs async, monolith vs services, consistency level, single vs multi-region), record the options, what you sacrificed, and the requirement that drove the choice. This log *is* your design's justification — and the story you'd tell in an interview.

Chapter summary

- Every design decision is a **trade-off** — consistency/availability, simplicity/scalability, cost/reliability, build/buy.
- Communicate by **thinking aloud**, structuring, drawing, and driving the discussion with alternatives.
- **Justify** each choice by options + trade-off + the **requirement** behind it.
- System design is the art of **well-reasoned trade-offs under constraints** — and explaining them clearly.

COMING NEXT — PART X

Chapters 55-70 — Case Studies. The method now gets applied end-to-end to real systems — a URL shortener, a social feed, chat, video streaming, ride-sharing, and more — each walked through the full framework from requirements to trade-offs.